Q-HEALTH GUARD

Qiskit Implementation Details

A beginner-friendly technical guide to quantum-kernel classification
for the privacy-preserving healthcare AI project

Project	Q-HEALTH GUARD
Team	Epic Engineers
Event	Qiskit Fall Fest 2026
Problem Statement ID	VNQFF-02

Implementation guide — experimental prototype, not a clinical diagnostic system.

1. What are we implementing?

The Qiskit component experiments with a **quantum-kernel classifier** to classify labels in the synthetic healthcare dataset. Classical patient features are scaled and encoded into a quantum circuit. A quantum kernel estimates similarity between pairs of encoded samples, and QSVC uses that kernel for classification.

Important: this is a quantum-machine-learning experiment. It does not prove quantum advantage, and it must not be used to diagnose or treat real patients.

2. Where Qiskit fits in Q-HEALTH GUARD

Stage	What happens
1. Dataset preparation	Load synthetic records, clean column names, filter to explicit research-consent = Yes for this prototype.
2. Feature preparation	Choose a small set of numeric features and scale them to the circuit's input range.
3. Quantum feature map	Encode the numeric values into a parameterized circuit; ZZFeatureMap includes feature-dependent rotatio
4. Quantum kernel	FidelityQuantumKernel compares encoded states to build a kernel matrix.
5. QSVC	A support-vector classifier learns from the kernel matrix and predicts held-out labels.
6. Evaluation	Report accuracy and per-class precision/recall/F1; compare against a classical baseline.

3. Key quantum terms

Qubits

A qubit is the basic unit of quantum information. In this prototype, the circuit uses four qubits because four numeric input features are selected. This is a design choice for a small demonstration, not a claim that each feature is literally a physical qubit in a real hospital.

Quantum circuit

A circuit is a sequence of quantum operations (gates) applied to qubits. The feature-map circuit transforms each scaled input row into a quantum state whose structure depends on the input values.

Gates used by the feature map

Hadamard (H): creates a superposition from a computational-basis state. **RZ rotations:** encode feature-dependent phases. **Entangling operations (often controlled-Z interactions):** correlate qubits so the representation can include relationships between features. The exact gate sequence depends on the Qiskit feature-map version and settings.

Quantum kernel

A kernel measures similarity between examples. FidelityQuantumKernel uses the overlap/fidelity between quantum states prepared from two inputs. Similarity values are assembled into a kernel matrix.

QSVC

QSVC is a support-vector classifier that uses a quantum kernel. It is not a quantum neural network and does not automatically implement federated learning.

4. Dataset and feature choices

The project dataset is synthetic and has 200 rows and 32 columns, including the target column **diagnosis_label** and the consent field **consent_for_research**. The proposed starter code uses four numerical features: **age_years, bmi, temperature_c, heart_rate_bpm**. Four features make a compact demo circuit.

For a conservative prototype, the code below filters to rows whose consent value is explicitly Yes. Because the dataset is synthetic, this is a workflow demonstration—not evidence of real patient consent or clinical validity. Check the dataset's actual usage terms.

The diagnosis labels are multiclass. With a small consent-filtered dataset, some labels may have too few examples for stratified splitting; the code checks this and provides a clear error instead of silently producing a misleading result.

5. Install dependencies

```
python -m pip install qiskit qiskit-machine-learning scikit-learn pandas numpy
```

Run this in the terminal for the Python environment that will run the app. If using Python IDLE, install packages into the same Python interpreter. Qiskit package APIs can change between releases.

6. Complete starter code

Save this as **qiskit_health_classifier.py** in the same folder as **synthetic_privacy_preserving_health_dataset.csv**. The code performs local quantum-kernel classification only; it does not yet implement federated aggregation or differential privacy.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
from sklearn.svm import SVC

from qiskit.circuit.library import zz_feature_map
from qiskit_machine_learning.kernels import FidelityQuantumKernel
from qiskit_machine_learning.algorithms import QSVC

# 1) Load and clean the dataset
df = pd.read_csv("synthetic_privacy_preserving_health_dataset.csv")
df.columns = df.columns.str.strip()

required = [
    "consent_for_research", "diagnosis_label",
    "age_years", "bmi", "temperature_c", "heart_rate_bpm"
]
missing = [col for col in required if col not in df.columns]
if missing:
    raise ValueError(f"Dataset is missing columns: {missing}")

# 2) Keep only explicit Yes rows for this prototype
df = df[
    df["consent_for_research"]
      .astype(str).str.strip().str.lower().eq("yes")
].copy()

features = ["age_years", "bmi", "temperature_c", "heart_rate_bpm"]
df = df.dropna(subset=features + ["diagnosis_label"])

if len(df) < 8:
    raise ValueError(
        "Too few consent-filtered rows. Check the dataset and consent filter."
    )

X = df[features].astype(float)
y_text = df["diagnosis_label"].astype(str)

# Ensure every class has enough examples for a stratified split
counts = y_text.value_counts()
if counts.min() < 2:
    raise ValueError(
        "At least one diagnosis class has fewer than 2 records after filtering. "
```

```
        "Use a larger suitable dataset or document a different split strategy."
    )

encoder = LabelEncoder()
y = encoder.fit_transform(y_text)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Fit scaler only on training data to avoid data leakage
scaler = MinMaxScaler(feature_range=(0, np.pi))
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3) Define a four-qubit quantum feature map
feature_map = zz_feature_map(
    feature_dimension=len(features),
    reps=2,
    entanglement="linear"
)

# 4) Build a fidelity-based quantum kernel and QSVC classifier
quantum_kernel = FidelityQuantumKernel(feature_map=feature_map)
quantum_model = QSVC(quantum_kernel=quantum_kernel)
quantum_model.fit(X_train_scaled, y_train)

# 5) Evaluate the quantum model
quantum_predictions = quantum_model.predict(X_test_scaled)
print("\\n=== Quantum-kernel QSVC ===")
print("Accuracy:", round(
    accuracy_score(y_test, quantum_predictions), 4
))
print(classification_report(
    y_test,
    quantum_predictions,
    labels=np.arange(len(encoder.classes_)),
    target_names=encoder.classes_,
    zero_division=0
))

# 6) Classical baseline using the same scaled data and split
classical_model = SVC(kernel="rbf")
classical_model.fit(X_train_scaled, y_train)
classical_predictions = classical_model.predict(X_test_scaled)

print("\\n=== Classical RBF-SVC baseline ===")
print("Accuracy:", round(
    accuracy_score(y_test, classical_predictions), 4
))
print(classification_report(
    y_test,
    classical_predictions,
    labels=np.arange(len(encoder.classes_)),
    target_names=encoder.classes_,
    zero_division=0
))
```

If your installed Qiskit version does not recognize `zz_feature_map` or the QSVC constructor, check the version-specific Qiskit Machine Learning documentation or share the full error message. Do not change APIs at random.

7. What the circuit settings mean

Setting	Meaning in this project
feature_dimension = 4	Four input values are encoded, so the feature map has four qubits.
reps = 2	Repeats the feature-map pattern twice, increasing circuit depth.
entanglement = linear	Entangling connections follow a line-like pattern rather than connecting every pair.
MinMaxScaler to 0..π	Maps training features into a bounded range suitable for the selected rotations.
FidelityQuantumKernel	Computes pairwise state similarities for training and prediction.

Setting	Meaning in this project
QSVC	Uses the quantum kernel in a support-vector classification workflow.

8. How to explain it in a presentation

Short explanation: “We select a few numerical features from a synthetic healthcare dataset and scale them. Qiskit's ZZ feature map encodes each row into a four-qubit circuit. FidelityQuantumKernel computes similarities between the encoded quantum states, and QSVC uses those similarities to predict the diagnosis label. We compare the result with a classical RBF-SVC baseline. This is an experimental quantum-kernel component; federated learning, differential privacy, and secure aggregation must be implemented and tested as separate components.”

9. How it relates to federated learning

A conventional FedAvg workflow assumes clients train compatible parameterized models and average their model parameters. A QSVC built around a kernel matrix does not naturally expose a common set of model weights to average. Therefore, do **not** claim that this QSVC code is already federated.

For a defensible architecture, present the QSVC as a local/experimental quantum classifier, and implement federated learning separately with a model designed for federated parameter aggregation. If the project specifically requires federated quantum learning, choose and document a trainable quantum model and an explicit aggregation method, then validate that design experimentally.

Differential privacy is not implemented by the code above. Clipping and adding calibrated noise to suitable updates, with privacy parameters and accounting, would be a separate engineering task. Federated learning alone does not guarantee privacy.

10. Limitations and checklist

• Dataset is synthetic; performance is not clinical evidence.

• Report the test split, class distribution, random seed, and both quantum and classical metrics.

• Do not claim quantum advantage unless supported by a fair, reproducible comparison.

• Kernel computation can become expensive as the number of samples increases.

• The starter code runs one local classifier; it does not implement a web dashboard, multi-hospital networking, federated aggregation, differential privacy, or secure aggregation.

• Do not use model predictions for real diagnosis or treatment decisions.

11. Suggested references

Qiskit Machine Learning documentation: quantum kernels and QSVC API. Qiskit documentation for feature maps and quantum circuits. Refer to documentation matching the versions installed in your environment.

End of implementation guide.